

Code review — java-abstractizare

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-07 — sha e4a8e74

Ai lucrat ambele fișe, `abstract` și `interface`, într-un singur commit: `Triunghi`, `Examen`, `Consultatii`, clasa intermediară `HolderSala`, desenul în desen, `Duplicabil`, `ArieFigura`, `Comparable` cu `Collections.sort`.

Poarta de compilare: **trece**. `javac` pe toate sursele iese cu 0 erori și `java app.App` pornește.

Ce e bine, și contează:

- **Formatul rândului e scris o singură dată.** `afisare()` din `Interval` e concretă și cheamă `getTipInterval()` și `detalii()`, care sunt abstracte. Ai adăugat `Consultatii` fără să atingi `Interval`. E exact piesa-cheie din teoria de `abstract`.
- `HolderSala` e intermediară și rămâne abstractă, iar constructorii din `Interval` sunt `protected`.
- `Desen.duplicare()` copiază recursiv: un desen din desen se copiază corect, până la ultimul punct.
- Aria triunghiului e corectă: triunghiul (0,0), (4,0), (0,3) dă 6.
- Din runda 2 de la moștenire ai închis **B4** (orarul cu un singur interval), **B7** (al doilea constructor al `Eticheta`) și **M5** (`getArrIntervale()` întoarce o copie).

Problema rundei: **ai mutat pachetul `OrarScheduler` în repo-ul nou cu patru** ● **din runda 2 de la moștenire, neatinse:** copierea care golește orarul, decalarea, ora afișată și `translate`. Runda 2 avea fix-ul scris pentru fiecare. Dar pe 2026-09-26 PDF-ul rundei 2 a fost înlocuit în repo-ul tău cu cel al rundei 1. Așa că le găsești mai jos din nou, marcate „**din runda 2**”.

Tot ce scriu mai jos e **reprodus prin rulare**, nu dedus.

Critice

B1 ● `Orar.duplicate()` întoarce un orar gol

`src/app/OrarScheduler/Orar.java:84–87` · același tipar la `Scheduler.java:36–38`

CUM E ACUM: un orar cu 2 intervale, copiat pe cele două căi pe care le ai:

```
duplicate():          0 intervale
interfaceDuplicate(): 2 intervale
```

DE CE: `copie.getArrIntervale()` nu-ți dă lista copieii. Îți dă **o listă nouă**, făcută chiar atunci (`new ArrayList<> (arrIntervale)`). Ai scris-o așa ca să nu poată modifica nimeni orarul din afară, și e corect. Doar că asta te oprește și pe

tine: `.add(...)` pune intervalul în lista temporară, iar lista temporară e aruncată la sfârșitul liniei. Copia rămâne cu lista ei goală.

Varianta corectă e chiar în fișierul tău, pe linia 86, comentată: `copie.adaugaInterval(interval.duplicate());`. Și `interfaceDuplicate()` de mai jos o folosește, de-aceea merge. Ai două metode care ar trebui să facă același lucru, și au ajuns să facă lucruri diferite. Asta e prețul duplicatului din M1.

`Scheduler.duplicate()` are aceeași construcție (`getListaOrare().add(...)`), dar acolo lista e `null`, deci crapă înainte să ajungă la `add`: `NullPointerException: ... because "this.listaOrare" is null`.

FIX: `copie.adaugaInterval(interval.duplicate());`, adică linia 86.

B2 Constructorul de copiere `Orar(Orar)` golește originalul (din runda 2, B1)

`src/app/OrarScheduler/Orar.java:20-22`

CUM E ACUM:

```
inainte, original: 2
dupa new Orar(o), original: 0, copia: 0
```

DE CE: în constructor, `this` e obiectul **nou**, iar `arrIntervale` e lista lui goală, tocmai creată la linia 13. `orar.setArrIntervale(arrIntervale)` înseamnă „pune lista mea goală în sursă”. E direcția inversă: sursa își pierde intervalele, iar copia rămâne goală. Nu-l cheamă nimeni acum, dar prima persoană care îl folosește pierde orarul.

FIX: o listă nouă în `this`, apoi `adaugaInterval(interval.duplicate())` pentru fiecare interval din sursă.

B3 `Orar.toString()` arată doar primul interval

`src/app/OrarScheduler/Orar.java:44-54`

CUM E ACUM: în `CrossClass`, un orar cu `PAUZA` și `CURS`:

```
Lista originale: [Cerc cu punctul: X:1,Y:1, raza: 2, PAUZA: ora 11:10]
```

`CURS` -ul nu apare nicăieri.

DE CE: `return` nu iese doar din buclă, iese din **toată metoda**. La prima trecere prin `for` funcția s-a terminat deja, cu primul interval. Din același motiv `return "null"` de la final nu se execută niciodată: compilatorul ți-a cerut un `return` după buclă, iar textul `"null"` e locul în care ai scris ceva doar ca să tacă.

Modelul corect e chiar în codul tău: `Desen.toString()` adună tot textul într-un `StringBuilder` și dă `return` o singură dată, **după** buclă.

FIX: la fel ca în `Desen`: aduni în buclă, dai `return` după ea.

B4 Decalarea pierde minutele (din runda 2, B2)

`src/app/OrarScheduler/Interval.java:28` și `:83-109`

CUM E ACUM:

```
CURS: ora 11:00 ...  
Orarul a fost decalat. Noua ora de incepere CURS este ora 10:030  
CURS: ora 11:00 ... | getOraInceput=10
```

Mesajul zice 10:30 (scris „10:030”), dar obiectul a rămas la 10. Un curs de la 23:00 decalat cu 90 de minute ajunge la 0:00, nu la 0:30.

DE CE: câmpul ține minute, dar `setOraInceput` primește **ore** și înmulțește cu 60. `decalare` calculează corect 630 de minute, apoi le împarte la 60 (rezultă 10) ca să le poată da setter-ului. Cele 30 de minute se pierd la împărțirea întreagă, chiar la salvare.

FIX: unitatea se alege o singură dată, la câmp. Setter-ul primește minute și `decalare` se reduce la `oraInceput = oraInceput + nrMinute`.

B5  **Afișarea arată ora de sfârșit, iar minutele de sub 10 ies fără zero (din runda 2, B3)**

`src/app/OrarScheduler/Interval.java:41-60` · același cod copiat în `:62-81`

CUM E ACUM:

```
new OraCurs(10, 60, ...) -> CURS: ora 11:00 (incepe la 10)  
new OraCurs(10, 65, ...) -> CURS: ora 11:5  
new OraCurs(10, 125, ...) -> CURS: ora 12:5
```

DE CE: `ora += getDurataMin() / 60` adună durata peste ora de început, deci obții sfârșitul. Ora de început, cea cerută, nu se afișează nicăieri.

Zeroul în față se pune pe o singură ramură, `else if (getDurataMin() < 10)`. Pe ramura de `>= 60` minutele pot fi tot sub 10 ($65 \% 60 = 5$), dar acolo nu verifică nimeni. E aceeași regulă („minut sub 10 = zero în față”), aplicată într-un loc și uitată în celălalt. Și fiindcă `toString()` e o copie a lui `afisare()` (M2), bug-ul există de două ori.

FIX: o singură bucată de cod care transformă un număr de minute în `hh:mm`, chemată pentru început și, dacă vrei, pentru sfârșit.

B6  **`Punct.translate` teleportează (din runda 2, B5)**

`src/app/formeGeometrice/Punct.java:66-70`

CUM E ACUM:

```
Punct(20,50).translate(100,100) -> X:100,Y:100  
Punct(30, 0).translate(100,100) -> X:100,Y:100
```

Două puncte diferite, mutate cu aceeași valoare, ajung în același loc. La exercițiul 1 din fișa de `abstract`, „Gata când” e chiar proba asta: cele 3 puncte ale triunghiului se mută cu aceeași valoare. Acum toate trei ajung în (100,100). Comentariul tău din `Desen`, `//muta tot desenul – teleport`, descrie exact ce se întâmplă.

DE CE: `setX(x)` **pune** valoarea primită în loc s-o adune. Toate figurile deleagă către `Punct.translate`, deci o singură linie greșită strică mutarea în toată ierarhia.

⚠ **Ce vei vedea după fix:** În `exFiguriGeometrice` același obiect `punct1` e pus și în triunghi, și în dreptunghi, și în cerc. Un `translate` pe desen îl va muta de **trei ori**. Acum nu se vede, pentru că teleportarea de trei ori în același loc dă tot același loc. Test pe un punct folosit în două figuri: `(0,0)` mutat cu `(7,7)` trebuie să ajungă în `(7,7)`, iar după fix va ajunge în `(14,14)`.

FIX: `this.x += x; this.y += y;` și câte un `new Punct(...)` separat pentru fiecare figură.

B7 🚫 `Triunghi.afisare()` nu afișează nimic

`src/app/formeGeometrice/Triunghi.java:54-56`

CUM E ACUM: `triunghi.afisare()` tipărește un rând gol: `[]`.

DE CE: `afisare()` e abstractă în `Figura`, deci compilatorul te-a obligat s-o scrii, și ai scris-o cu corpul gol. Ce se vede de aici: `abstract` te poate obliga să ai metoda, dar nu poate verifica ce faci în ea. E fix semnul din finalul teoriei de `abstract`: „corpul gol e semnul”. Aici corpul gol e în clasa concretă, unde nimeni nu mai vine după tine să-l scrie.

Nu se observă în desen fiindcă `Desen` afișează prin `toString()`, nu prin `afisare()`.

FIX: `System.out.println(this);`, ca la celelalte figuri.

B8 🚫 `adunaAriile()` ignoră desenul din desen

`src/app/formeGeometrice/Desen.java:64-75`

CUM E ACUM:

```
casa (dreptunghi 10x10 + triunghi cu arie 6): 106.0
mare (casa + dreptunghi 1x1): 1.0
```

Aria desenului mare trebuia să fie 107.

DE CE: `instanceof ArieFigura` întreabă dacă obiectul a promis că are arie. `Desen` nu a promis (nu are `implements ArieFigura`), deci casa e trecută cu vederea în tăcere, ca și cum ar fi un `Punct`. Desenul în desen merge la afișare, mutare și copiere, pentru că acolo `Desen` are metodele din `Figura`. La arie nu le are.

Și, ca în rundele trecute, metoda nu e chemată nicăieri în proiect. Dacă ai fi rulat-o o dată pe `desenMare`, din `App`, ai fi văzut aria cercului lipsă.

FIX: `Desen implements ArieFigura`, iar `calculeazaArie()` din `Desen` e chiar suma de acum. Cu asta, un desen interior intră în sumă singur.

Importante

M1 🟡 Două metode de copiere în fiecare clasă

17 fișiere au `interfaceDuplicate()` pe lângă `duplicare()` / `duplicate()`, cu același corp. Exercițiul 1 de la `interface` cerea ca toate trei ierarhiile să-și copieze obiectele printr-o singură metodă, cu un singur nume. B1 arată de ce: cele două copii ale lui `Orar` au ajuns deja să facă lucruri diferite.

M2 ● `toString()` din `Interval` e o copie a lui `afisare()`

`Interval.java:41-60` și `:62-81` au aceleași 20 de rânduri. Una dintre metode trebuie s-o cheme pe cealaltă. Altfel, un fix făcut într-una (B5) lasă bug-ul în cealaltă.

M3 ● Declarații repetate

- `HolderSala.java:31-37` redeclară cele 4 metode abstracte din `Interval`. La întrebarea ta de pe linia 38, `//mai punem duplicate si aici?` : **nu**. Metoda abstractă se moștenește tot abstractă, așa că `HolderSala` n-are nevoie să o mai scrie.
- `implements Duplicabil` e scris și pe `Pauza`, `Examen`, `Consultatii`, `ActivitateLibera` și pe toate figurile, deși `Interval` și `Figura` îl au deja. Ce implementează clasa-părinte moștenesc și subclasele.

M4 ● `new Orar(lista)` lucrează pe lista celui care l-a creat

`Orar.java:24-26` : după `new Orar(l).sorteazaIntervale()`, lista `l` din `App` e și ea sortată. Orarul nu are o listă a lui, o folosește pe cea primită. Același principiu ca la `getArrIntervale()`, pe care l-ai rezolvat corect: o listă **nouă** atât la intrare, cât și la ieșire.

M5 ● `Scheduler` e tot o clasă moartă (din runda 2, M2)

`creazaOrar()` construiește un orar și îl aruncă, `listaOrare` nu e niciodată inițializată, iar `duplicate()` crapă (vezi B1).

M6 ● Exerciții sărite

- `abstract` ex. 7 și `interface` ex. 2: lipsește proba cu `Romb`.
- `interface` ex. 5: `Eticheta` tot moștenește `Dreptunghi` și ține un `Dreptunghi` (din runda 2, M3).
- `interface` ex. 6 (`Notiță`) și ex. 7 (argumentul pentru `Figura`): lipsesc.

Cleanups

- C1** ● `println` -uri de test rămase: `compareTo` tipărește „12 vs 14” la fiecare comparație (9 rânduri la o sortare de 5), `sorteazaIntervale` tipărește „test”, iar `decalare` scrie „0test” în mesaj.
- C2** ● `Main.java` e șablonul IntelliJ („Hello and welcome!”). Proiectul are două `main`, iar cel adevărat e în `App`.
- C3** ● Importuri care nu folosesc la nimic: `Duplicabil.java` importă `Interval`, `CrossClass.java` importă `app.Duplicabil` (e chiar în același pachet), `Orar.java` importă `Collection`.
- C4** ● Numele de pachete în Java se scriu cu litere mici: `orarscheduler`, nu `OrarScheduler`.

Întrebarea ta din cod

`App.java:116` : `arrDuplicate.addAll(Arrays.asList(cerc1, curs1, orar1)); //???`

Merge pentru că toate trei **au promis** `Duplicabil`. `Cerc` îl are prin `Figura`, `OraCurs` prin `Interval`, iar `Orar` direct. Lista nu vrea să știe ce **sunt** obiectele, doar ce **pot face**. Țasta e tot rostul exercițiului 1: un `Orar` și un `Cerc` n-au niciun părinte comun, dar au o capacitate comună.

Before / after (critice)

#	Cum e acum	Cum ar trebui
B1	<code>copie.getArrIntervale().add(interval.duplicate());</code>	<code>copie.adaugaInterval(interval.duplicate());</code>
B2	<code>orar.setArrIntervale(arrIntervale);</code>	bucă peste sursă cu <code>adaugaInterval(interval.duplicate())</code>
B3	<code>return interval.afisare();</code> în <code>for</code>	<code>StringBuilder</code> în buclă, <code>return</code> după ea
B4	<code>setOraInceput(oraDecalata)</code> după <code>/ 60</code>	câmpul în minute: <code>oraInceput = oraInceput + nrMinute;</code>
B5	<code>ora += getDurataMin() / 60</code> , zero doar pe o ramură	o singură metodă minute → <code>hh:mm</code> , pentru început
B6	<code>this.setX(x); this.setY(y);</code>	<code>this.x += x; this.y += y;</code>
B7	<code>public void afisare(){ }</code>	<code>System.out.println(this);</code>
B8	<code>class Desen extends Figura implements Duplicabil</code>	<code>... implements Duplicabil, ArieFigura</code>

Q&A — răspunde în scris, două-trei rânduri fiecare

1. **B1.** `getArrIntervale()` întoarce o copie ca să nu poată nimeni din afară să strice orarul. Cum ai scrie atunci, din afara clasei `Orar`, „adaugă un interval în orarul ăsta”? Și de ce `adaugaInterval` nu are aceeași problemă?
2. **B7 + B8.** Compilatorul te-a obligat să scrii `afisare()` în `Triunghi`, dar nu te-a obligat să scrii `calculeazaArie()` în `Desen`. De ce diferența? Care dintre cele două cuvinte, `abstract` sau `implements`, te „prinde” și când?
3. **Runda 2.** Patru dintre bug-urile de aici aveau fix-ul scris în runda 2 de la moștenire. Ai citit runda 2? Dacă nu, care a fost motivul: n-ai văzut PDF-ul, sau l-ai deschis și n-ai înțeles ce cere?